

APPLIED DATA STRUCTURES & ALGORITHMS LAB (BCS-553)

B.Tech CSE 5th SEMESTER

SUBMMITED TO:

MR. RASHAN RAJ

SUBMMITED BY:

MANKIRAT SINGH

ID: BCS2024542

ROLL NO: 2410303257

Lab Assignment 6: Binary Search on a Sorted Array

Experiment No. 6.1 - Binary search for key 40

Experiment Title:

Binary search for key 40

Aim:

To search for the key 40 in a sorted array using binary search.

Software / Tools Required:

Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

Theory:

Binary search works only on a sorted array. It compares the key with the middle element; if the key is smaller it continues in the left half, if larger in the right half. Because the search range halves every step, the time complexity is O(log n).

Algorithm / Procedure:

1. Set low = 0 and high = length - 1.
2. While low <= high, compute mid = low + (high - low) / 2.
3. If a[mid] == key return mid; if a[mid] < key set low = mid + 1; otherwise set high = mid - 1.
4. If the loop ends, return -1.

Program / Queries / Commands:

```
// Name: Mankirat Singh
// ID: BCS2024542
// Roll No: 24103032571
// Assignment 6, Task 1
// Question: Write the method int binarySearch(int[] a, int key) for
// a sorted array, returning the index of key or -1. Call it on {10,
// 20, 30, 40, 50} with key 40 and print the result.

public class Main {
    static int binarySearch(int[] a, int key) {
        int low = 0;
        int high = a.length - 1;

        while (low <= high) {
            int mid = low + (high - low) / 2;

            if (a[mid] == key) {
                return mid;
            } else if (a[mid] < key) {
                low = mid + 1;
            } else {
```

```
                high = mid - 1;
            }
        }
        return -1;
    }

    public static void main(String[] args) {
        int[] a = {10, 20, 30, 40, 50};
        System.out.println(binarySearch(a, 40));
    }
}
```

Input:

Sorted array {10, 20, 30, 40, 50}, key = 40.

Output:

```
3
```

Time / Space Complexity: Time O(log n), best case O(1); Space O(1).

Result:

The program was compiled and executed successfully. The key 40 was found at index 3.

Viva-Voce Questions:

- Why must the array be sorted for binary search?
- What is the time complexity of binary search?
- Why is mid computed as low + (high - low) / 2?

Experiment No. 6.2 - Binary search for key 35 (absent)

Experiment Title:

Binary search for key 35 (absent)

Aim:

To show that binary search returns -1 when the key is not present.

Software / Tools Required:

Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

Theory:

Binary search works only on a sorted array. It compares the key with the middle element; if the key is smaller it continues in the left half, if larger in the right half. Because the search range halves every step, the time complexity is O(log n).

Algorithm / Procedure:

1. Set low = 0 and high = length - 1.
2. While low <= high, compute mid = low + (high - low) / 2.
3. If a[mid] == key return mid; if a[mid] < key set low = mid + 1; otherwise set high = mid - 1.
4. If the loop ends, return -1.

Program / Queries / Commands:

```
// Name: Mankirat Singh
// ID: BCS2024542
// Roll No: 24103032571
// Assignment 6, Task 2
// Question: Write the method int binarySearch(int[] a, int key) for
// a sorted array, returning the index of key or -1. Call it on {10,
// 20, 30, 40, 50} with key 35 and print the result.

public class Main {
    static int binarySearch(int[] a, int key) {
        int low = 0;
        int high = a.length - 1;

        while (low <= high) {
            int mid = low + (high - low) / 2;

            if (a[mid] == key) {
                return mid;
            } else if (a[mid] < key) {
                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }
        return -1;
    }
```

```
    public static void main(String[] args) {
        int[] a = {10, 20, 30, 40, 50};
        System.out.println(binarySearch(a, 35));
    }
}
```

Input:

Sorted array {10, 20, 30, 40, 50}, key = 35.

Output:

```
-1
```

Time / Space Complexity: Time O(log n); Space O(1).

Result:

The program was compiled and executed successfully. The key 35 is not in the array, so the method returned -1.

Viva-Voce Questions:

- When does the while loop stop for an absent key?
- How many steps are needed for 35 here?
- What is returned for a missing key?

Experiment No. 6.3 - Binary search for key 10 (first element)

Experiment Title:

Binary search for key 10 (first element)

Aim:

To search for the first element of a sorted array using binary search.

Software / Tools Required:

Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

Theory:

Binary search works only on a sorted array. It compares the key with the middle element; if the key is smaller it continues in the left half, if larger in the right half. Because the search range halves every step, the time complexity is O(log n).

Algorithm / Procedure:

1. Set low = 0 and high = length - 1.
2. While low <= high, compute mid = low + (high - low) / 2.
3. If a[mid] == key return mid; if a[mid] < key set low = mid + 1; otherwise set high = mid - 1.
4. If the loop ends, return -1.

Program / Queries / Commands:

```
// Name: Mankirat Singh
// ID: BCS2024542
// Roll No: 24103032571
// Assignment 6, Task 3
// Question: Write the method int binarySearch(int[] a, int key) for
// a sorted array, returning the index of key or -1. Call it on {10,
// 20, 30, 40, 50} with key 10 and print the result.

public class Main {
    static int binarySearch(int[] a, int key) {
        int low = 0;
        int high = a.length - 1;

        while (low <= high) {
            int mid = low + (high - low) / 2;

            if (a[mid] == key) {
                return mid;
            } else if (a[mid] < key) {
                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }
        return -1;
    }
```

```
    public static void main(String[] args) {
        int[] a = {10, 20, 30, 40, 50};
        System.out.println(binarySearch(a, 10));
    }
}
```

Input:

Sorted array {10, 20, 30, 40, 50}, key = 10.

Output:

```
0
```

Time / Space Complexity: Time O(log n); Space O(1).

Result:

The program was compiled and executed successfully. The key 10 was found at index 0.

Viva-Voce Questions:

- How does the search range change for key 10?
- Which is faster for large arrays: linear or binary search?
- Can binary search be written recursively?